

Project 2: Unix utilities – documentation

Student: Lauri Metso

Repository: <https://github.com/laurimetso/systeemiohjelmointi>

Summary

This project implements four simplified UNIX command-line utilities in C: my-cat, my-grep, my-zip and my-unzip. Each program is contained in their own .c source file and compiles with the -Wall -Werror flags using gcc.

- my-cat: Reads one or more files and prints their contents to stdout
- my-grep: Searches files (or stdin) for a term and prints matching lines
- my-zip: Compresses files using run-length encoding
- my-unzip: Decompresses files produced by my-zip

All programs return 0 on success and exit with status 1 on error, printing an appropriate error message.

How to compile

```
gcc my-cat.c -o my-cat -Wall -Werror
```

```
gcc my-grep.c -o my-grep -Wall -Werror
```

```
gcc my-zip.c -o my-zip -Wall -Werror
```

```
gcc my-unzip.c -o my-unzip -Wall -Werror
```

Program descriptions

my-cat

Reads one or more files and prints their contents to standard output in order.

Usage:

```
./my-cat file1 [file2 ...]
```

Implementation:

If no files are given (`argc == 1`), the program exits immediately with status 0. Otherwise it iterates over the command-line arguments, opening each file with `fopen()` in read mode. If a file cannot be opened, the program prints "my-cat: cannot open file" and exits with status 1.

File contents are read into a 4096-byte buffer using `fgets()` and printed with `printf("%s", buffer)`, which preserves newlines exactly as they appear in the file. Each file is closed with `fclose()` after reading.

Note: lines longer than 4095 characters are still printed correctly `fgets()` will simply be called again and continue from where it left off. This can be tested by creating a file with 10000 letters, and then doing `./my-cat examplelong.txt | wc -c` on it. It will print 10001 (10000 characters + newline).

Example:

```
./my-cat main.c
```

```
./my-cat file1.txt file2.txt
```

my-grep

Searches files (or stdin) line by line and prints every line containing the search term. Search is case-sensitive.

Usage:

```
./my-grep searchterm [file1 file2 ...]
```

Implementation:

If no arguments are given, the program prints the usage message and exits with status 1. If only a search term is given with no files, my-grep reads from stdin, making it usable in shell pipelines. The core logic lives in the helper function `grep_file()`, which takes a search term and an open FILE pointer. It reads lines with `getline()` which allocates memory dynamically and handles arbitrarily long lines and uses `strstr()` to check for the search term. Matching lines are printed with `printf()`. The line buffer is freed with `free()` when done.

For each file argument, `fopen()` is called and its result checked; if a file cannot be opened, the program prints "my-grep: cannot open file" and exits with status 1.

Example:

```
./my-grep foo bar.txt
```

```
./my-grep error logfile1.txt logfile2.txt
```

```
echo "hello world" | ./my-grep hello
```

my-zip

Compresses one or more files using run-length encoding (RLE). Consecutive identical characters are stored as a 4-byte integer count followed by the character. Output goes to stdout and should be redirected to a file.

Usage:

```
./my-zip file1 [file2 ...] > compressed.z
```

Implementation:

If no files are given, the program prints the usage message and exits with status 1. Each file is opened and passed to the helper function `zip_file()`. `zip_file()` reads characters one at a time with `fgetc()`, tracking the current character (`prev`) and its consecutive count. When the character changes, or when the count reaches 255 (to avoid overflow), the accumulated run is written out: first the 4-byte count using `fwrite(&count, 4, 1, stdout)`, then the character with `fputc()`. The final run is flushed after the loop ends.

Note: because `zip_file()` is called separately per file, runs do not span file boundaries; each file is compressed independently into the stream.

Example:

```
./my-zip report.txt > report.z
```

```
./my-zip file1.txt file2.txt > combined.z
```

my-unzip

Decompresses files produced by my-zip, writing the original content to standard output.

Usage:

```
./my-unzip compressed.z
```

Implementation:

If no files are given, the program prints the usage message and exits with status 1. Each file is opened in binary mode ("rb") and passed to the helper function `unzip_file()`.

`unzip_file()` reads 5-byte entries in a loop using `fread()`: first a 4-byte integer (the run count), then a 1-byte character. If the character read fails after a successful count read, the loop breaks to handle malformed input gracefully. The character is then printed count times with `printf()`. Opening in binary

mode ("rb") is important -- on some systems, text mode would translate line endings and corrupt the binary compressed format.

Example:

```
./my-unzip report.z
```

```
./my-unzip combined.z > restored.txt
```

Scored features

Feature	Score
Works	Required, 1
Documentation	Required, 1
My-cat	1
My-grep	1
My-zip and unzip	1

Notes

Run count cap in my-zip: `zip_file()` resets the run when count reaches 255. This prevents silent overflow and keeps individual run lengths at a safe maximum size.

Binary mode in my-unzip:

my-unzip opens files with "rb" so `fread()` reads raw bytes without any newline translation, which is required to parse the compressed format correctly.

Arbitrary line length in my-grep: `getline()` is used instead of `fgets()` so lines of any length are handled correctly, satisfying the specification's requirement.

stdin fallback in my-grep: When no file arguments are given, `grep_file()` is called with `stdin` directly. Since `stdin` is already open, no `fopen()` is needed and the program works naturally in shell pipelines.

Screenshots

```
GNU nano 6.2 my-cat.c
/*
 * my-cat.c
 * simple cat utility
 * reads and prints contents of a given file
 */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    // no files are provided, exit successfully
    if (argc == 1){
        return 0;
    }

    char buffer[4096];

    // process each file given in command-line arguments
    for (int i = 1; i < argc; i++) {
        // open file in read mode
        FILE *fp = fopen(argv[i], "r");

        // exit with error if file cant be opened
        if (fp == NULL) {
            printf("my-cat: cannot open file\n");
            exit(1);
        }

        // read and print file contents line by line
        while (fgets(buffer, sizeof(buffer), fp) != NULL){
            printf("%s", buffer);
        }

        // close file
        fclose(fp);
    }
    return(0);
}
```

```
#include <stdlib.h>
#include <string.h>

/*
 * reads lines from a file stream and prints
 * lines containing the search term
 */
void grep_file(char *searchterm, FILE *fp)
{
    char *line = NULL;
    size_t len = 0;
    // read file line by line
    while (getline(&line, &len, fp) != -1) {

        // print if matches
        if (strstr(line, searchterm) != NULL) {
            printf("%s", line);
        }
        free(line);
    }
}

int main(int argc, char *argv[])
{
    // must provide search term
    if (argc < 2) {
        printf("my-grep: searchterm [file ...]\n");
        exit(1);
    }

    char *searchterm = argv[1];

    // no file provided: read from standard input
    if (argc == 2) {
        grep_file(searchterm, stdin);
        return 0;
    }

    // process files
    for (int i = 2; i < argc; i++) {
        FILE *fp = fopen(argv[i], "r");

        // exit if file cant be opened
        if (fp == NULL) {
            printf("my-grep: cannot open file\n");
            exit(1);
        }

        grep_file(searchterm, fp);

        fclose(fp);
    }

    return 0;
}
```

```
GNU nano 6.2 my-zip.c
/*
 compresses a file using run-length encoding by storing
 consecutive characters as a count and the character itself.
*/

#include <stdio.h>
#include <stdlib.h>

void zip_file(FILE *fp)
{
    int character;
    int count = 1;
    int prev = EOF;

    while ((character = fgetc(fp)) != EOF) {
        // if character changed or count would overflow
        if (character != prev || count == 255) {
            if (prev != EOF) {
                // write the 4 byte count and the character
                fwrite(&count, 4, 1, stdout);
                fputc(prev, stdout);
            }
            prev = character;
            count = 1;
        } else {
            count++;
        }
    }
    // write the last run
    if (prev != EOF) {
        fwrite(&count, 4, 1, stdout);
        fputc(prev, stdout);
    }
}

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("my-zip: file1 [file2 ...] \n");
        exit(1);
    }

    // process each file
    for (int i = 1; i < argc; i++) {
        FILE *fp = fopen(argv[i], "r");
        if (fp == NULL) {
            printf("my-zip: cannot open file\n");
            exit(1);
        }
        zip_file(fp);
        fclose(fp);
    }
    return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace  ^U Paste    ^J Justify  ^_ Go To Line
```

```
GNU nano 6.2 my-unzip.c
/*
 creates normal text into a file from a run-length encoded file.
*/

#include <stdio.h>
#include <stdlib.h>

void unzip_file(FILE *fp)
{
    int count;
    char c;

    // read 5-byte entries: 4-byte int + 1-byte char
    while (fread(&count, 4, 1, fp) == 1) {
        if (fread(&c, 1, 1, fp) != 1) {
            break; // malformed file
        }
        // print character count times
        for (int i = 0; i < count; i++) {
            printf("%c", c);
        }
    }
}

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("my-unzip: file1 [file2 ...]\n");
        exit(1);
    }

    // process each file
    for (int i = 1; i < argc; i++) {
        FILE *fp = fopen(argv[i], "rb"); // binary mode
        if (fp == NULL) {
            printf("my-unzip: cannot open file\n");
            exit(1);
        }
        unzip_file(fp);
        fclose(fp);
    }

    return 0;
}

[ Read 44 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace  ^U Paste    ^J Justify  ^_ Go To Line
```